Gibbs Sampler Prototype: Marginalised Block Update, Adaptive, Anisotropic SPDE

Contents

Model **1**

Gibbs and marginalised Metropolis updates **2**

Update of μ . . . 2

Marginalised Metropolis–Hastings for the SPDE hyperparameters . . . 3

Setup and data generation **5**

MCMC initialisation **6**

Main Gibbs loop **7**

Convergence diagnostics **9**

Trace plots . . . 9

Autocorrelation and effective sample size . . . 10

Posterior correlation between hyperparameters . . . 11

Posterior estimates vs ground truth **12**

Estimated anomaly field . . . 13

Posterior mean and accuracy metrics . . . 14

Estimated vs true values . . . 14

Accuracy by year . . . 15

Maps for one year . . . 15

Model

For each year $t = 1, \ldots, M$ we observe the field at the N mesh nodes:

$$Y_t = \mu + \beta_t + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, \sigma^2 I_N)$$

- μ is the long-term mean field, with prior $\mu \sim \mathcal{N}(\mu_0, Q_\mu^{-1})$.
- β_t is the zero-mean spatial anomaly of year t, with prior $\beta_t \sim \mathcal{N}(0, Q_\beta^{-1})$ independently over years.

- Q_β is the anisotropic SPDE precision matrix built from the FEM matrices:

$$Q_\beta = \tau^2 \left(\kappa^2 C + G_H\right) C^{-1} \left(\kappa^2 C + G_H\right), \qquad G_H = H_{11} K_{11} + H_{12} K_{12} + H_{22} K_{22}$$

The anisotropy matrix H has ratio γ and rotation angle θ:

$$H_{11} = \gamma \cos^2\theta + \gamma^{-1} \sin^2\theta, \quad H_{22} = \gamma \sin^2\theta + \gamma^{-1} \cos^2\theta, \quad H_{12} = (\gamma - \gamma^{-1}) \sin\theta \cos\theta$$

Sampling scheme. Each iteration does the following:

1. Gibbs update of $\mu \mid \beta, \sigma^2, Y$.
2. Gibbs update of $\sigma^2 \mid \mu, \beta, Y$ (inverse-gamma).
3. Adaptive random-walk Metropolis–Hastings on $(\log\tau^2, \log\kappa^2, \log\gamma, \theta)$, using the likelihood **with β integrated out**, $p(Y - \mu \mid \tau^2, \kappa^2, \gamma, \theta, \sigma^2)$.
4. Joint draw of all β_t from their Gaussian full conditional, given the new hyperparameters.

Integrating β out in step 3 avoids the strong dependence between the hyperparameters and the latent field that makes a naive Gibbs sampler mix poorly.

> **Known issue: identifiability.** (γ, θ) and $(1/\gamma, \theta + \pi/2)$ give the same H, so the posterior has two equivalent modes. Fix it by constraining $\gamma \geq 1$ or $\theta \in [0, \pi/2)$.

Gibbs and marginalised Metropolis updates

Update of μ

The full conditional of μ is Gaussian. The data precision is M/σ^2 per node, and the data mean is the residual $Y - \beta$ averaged over years.

```
update_mu <- function(
  N,
  M,
  mean_mu_old,
  mean_q_old,
  Y_mat,
  curr_beta,
  curr_sigma2
) {
  q_data <- Diagonal(N, M / curr_sigma2)
  # Data contribution to mu is the residual after removing beta
  mean_data <- 1 / M * rowSums(Y_mat - curr_beta)

  new_q <- q_data + mean_q_old
  rhs <- as.vector(q_data %*% mean_data + mean_q_old %*% mean_mu_old)

  chol_factor <- chol(new_q)
  new_mean <- as.vector(solve(chol_factor, solve(t(chol_factor), rhs)))

  new_mu <- sample_normal_vector(new_mean, new_q, is_precision = TRUE)
  return(new_mu)
}
```

Marginalised Metropolis–Hastings for the SPDE hyperparameters

With $b_t = (Y_t - \mu)/\sigma^2$ and $Q_{\text{post}} = Q_\beta + \sigma^{-2} I$, the log marginal likelihood, up to terms that don't depend on the hyperparameters, is

$$\log p(Y \mid \cdot) = \frac{M}{2} \log |Q_\beta| - \frac{M}{2} \log |Q_{\text{post}}| + \frac{1}{2} \sum_t b_t^\top Q_{\text{post}}^{-1} b_t$$

Both log-determinants come from the Cholesky factors. The proposal is a Gaussian random walk on the unconstrained scale $(\log \tau^2, \log \kappa^2, \log \gamma, \theta)$, and the priors are $\mathcal{N}(0, 5^2)$ on the three logs.

```
marginal_metropolis_hastings <- function(
  curr_tau2,
  curr_kappa2,
  curr_gamma,
  curr_theta,
  B_mat,
  curr_sigma2,
  C_mat,
  K11,
  K22,
  K12,
  diag_C,
  prop_cov,
  Q_pr_curr,
  chol_pr_curr,
  chol_post_curr,
  M_post_curr,
  log_lik_curr
) {
  M_years <- ncol(B_mat)
  N_nodes <- nrow(B_mat)

  # 1. Proposal in unconstrained space
  curr_vec <- c(log(curr_tau2), log(curr_kappa2), log(curr_gamma), curr_theta)
  prop_vec <- sample_normal_vector(
    mean = curr_vec,
    mat = prop_cov,
    N = 4,
    is_precision = FALSE
  )

  tau2_prop <- exp(prop_vec[1])
  kappa2_prop <- exp(prop_vec[2])
  gamma_prop <- exp(prop_vec[3])
  theta_prop <- prop_vec[4] %% pi

  # 2. Build Proposed SPDE Matrices
  H11 <- gamma_prop *
    (cos(theta_prop)^2) +
    (1 / gamma_prop) * (sin(theta_prop)^2)
  H22 <- gamma_prop *
    (sin(theta_prop)^2) +
    (1 / gamma_prop) * (cos(theta_prop)^2)
```

```
H12 <- (gamma_prop - 1 / gamma_prop) * sin(theta_prop) * cos(theta_prop)

G_H <- H11 * K11 + H12 * K12 + H22 * K22
core_prop <- kappa2_prop * C_mat + G_H
Q_pr_prop <- forceSymmetric(
  tau2_prop * core_prop %*% (Diagonal(x = 1 / diag_C) %*% core_prop)
)
Q_post_prop <- Q_pr_prop + Diagonal(N_nodes, 1 / curr_sigma2)

# 3. Factorizations
chol_pr_prop <- chol(Q_pr_prop)
chol_post_prop <- chol(Q_post_prop)
M_post_prop <- solve(chol_post_prop, solve(t(chol_post_prop), B_mat))

# 4. MARGINAL Log-Likelihood
log_det_pr_prop <- 2 * sum(log(diag(chol_pr_prop)))
log_det_post_prop <- 2 * sum(log(diag(chol_post_prop)))
quad_prop <- sum(B_mat * M_post_prop)

log_lik_prop <- 0.5 *
  M_years *
  log_det_pr_prop -
  0.5 * M_years * log_det_post_prop +
  0.5 * quad_prop

# 5. Log-Priors
log_prior_curr <- dnorm(log(curr_tau2), 0, 5, log = TRUE) +
  dnorm(log(curr_kappa2), 0, 5, log = TRUE) +
  dnorm(log(curr_gamma), 0, 5, log = TRUE)
log_prior_prop <- dnorm(prop_vec[1], 0, 5, log = TRUE) +
  dnorm(prop_vec[2], 0, 5, log = TRUE) +
  dnorm(prop_vec[3], 0, 5, log = TRUE)

# 6. Accept / Reject
accepted <- FALSE
if (
  log(runif(1)) <
    (log_lik_prop + log_prior_prop - log_lik_curr - log_prior_curr)
) {
  curr_tau2 <- tau2_prop
  curr_kappa2 <- kappa2_prop
  curr_gamma <- gamma_prop
  curr_theta <- theta_prop

  Q_pr_curr <- Q_pr_prop
  chol_pr_curr <- chol_pr_prop
  chol_post_curr <- chol_post_prop
  M_post_curr <- M_post_prop
  log_lik_curr <- log_lik_prop
  accepted <- TRUE
}

return(list(
```

```
    tau2 = curr_tau2,
    kappa2 = curr_kappa2,
    gamma = curr_gamma,
    theta = curr_theta,
    Q_pr = Q_pr_curr,
    chol_pr = chol_pr_curr,
    chol_post = chol_post_curr,
    M_post = M_post_curr,
    log_lik = log_lik_curr,
    accepted = accepted
  ))
}
```

Setup and data generation

We simulate data on a 20×20 grid over $[0, 5]^2$ for $M = 10$ years, with known true parameters. The data generator returns $Y_t = \beta_t + \varepsilon_t$, where β_t has mean μ, so the true **anomaly** is $\beta_t - \mu$.

```
set.seed(42)
x <- seq(0, 5, length.out = 20)
y <- seq(0, 5, length.out = 20)
locs <- as.matrix(expand.grid(x = x, y = y))

mesh <- fm_mesh_2d(loc = locs)
N <- mesh$n
M <- 10
cat("Nr. of vertices (N):", N, "| Years (M):", M, "\n")
```

```
## Nr. of vertices (N): 436 | Years (M): 10
```

```
matrices <- build_spde_matrices(mesh, manifold = "planar")
K11 <- matrices$K11
K22 <- matrices$K22
K12 <- matrices$K12
C_mat <- matrices$C
diag_C <- Matrix::diag(C_mat)

true_tau <- 1.0
true_kappa <- 1.0
true_gamma <- 2.0
true_theta <- pi / 4
true_sigma_e <- sqrt(0.25)

sim <- generate_toy_data(
  mesh = mesh,
  num_year = M,
  mu_prior_mean = 0.5,
  Q_mu = Diagonal(N, 1),
  tau = true_tau,
  k = true_kappa,
  gamma = true_gamma,
```

```
  theta = true_theta,
  sigma_e = true_sigma_e
)

Y_mat <- sim$y_mat
mu_true <- sim$mu_true
# True zero-mean spatial anomaly
true_anomaly_mat <- sim$beta_mat - sim$mu_true
```

MCMC initialisation

- Run settings: $G = 5000$ iterations with a burn-in of 1000.
- Priors: $\sigma^2 \sim \text{IG}(0.01, 0.01)$ and $\mu \sim \mathcal{N}(0, 100I)$.
- Proposal: the covariance starts at $0.05^2 I$ and is adapted every 10 iterations after the first 200, using the scaled empirical covariance of the chain history, with the factor $2.38^2/d$ shrunk by 0.7.
- Stored draws: a thinned set of β draws (every 4th post-burn-in iteration), used for the validation in Section 8.

```
G <- 5000
burnin <- 1000
idx_post <- (burnin + 1):G
samples_sigma2 <- numeric(G)
samples_tau2 <- numeric(G)
samples_kappa2 <- numeric(G)
samples_gamma <- numeric(G)
samples_theta <- numeric(G)
samples_beta1 <- numeric(G)

curr_mu <- rep(0, N)
curr_beta <- matrix(0, nrow = N, ncol = M) # Zero-mean starting field
curr_sigma2 <- 1.0
curr_tau2 <- 0.5
curr_kappa2 <- 0.5
curr_gamma <- 1.0
curr_theta <- 0.0

prior_ig_a <- 0.01
prior_ig_b <- 0.01
prior_mu_prec <- Diagonal(N, 1 / 100)
prior_mu_mean <- rep(0, N)

H11_init <- curr_gamma *
  (cos(curr_theta)^2) +
  (1 / curr_gamma) * (sin(curr_theta)^2)
H22_init <- curr_gamma *
  (sin(curr_theta)^2) +
  (1 / curr_gamma) * (cos(curr_theta)^2)
H12_init <- (curr_gamma - 1 / curr_gamma) * sin(curr_theta) * cos(curr_theta)
G_H_init <- H11_init * K11 + H12_init * K12 + H22_init * K22

core_mat <- curr_kappa2 * C_mat + G_H_init
Q_pr_curr <- forceSymmetric(
```

```
  curr_tau2 * core_mat %*% (Diagonal(x = 1 / diag_C) %*% core_mat)
)
chol_pr_curr <- chol(Q_pr_curr)

mcmc_history <- matrix(0, nrow = G, ncol = 4)
colnames(mcmc_history) <- c("log_tau2", "log_kappa2", "log_gamma", "theta")
prop_cov <- diag(x = 0.05^2, nrow = 4)
scale_factor <- (2.38^2) / 4 * 0.7
acc_count <- 0

beta_sum_post <- matrix(0, nrow = N, ncol = M)

# Thinned storage of post burn-in beta draws (for validation)
thin <- 4
keep_iter <- idx_post[idx_post %% thin == 0]
beta_draws <- array(NA_real_, dim = c(N, M, length(keep_iter)))
```

Main Gibbs loop

This takes about 1.5 minutes for 436 nodes and 5000 iterations.

```
start_time <- Sys.time()
for (g in 1:G) {
  # Step 1: Update long-term mean mu
  curr_mu <- update_mu(
    N,
    M,
    prior_mu_mean,
    prior_mu_prec,
    Y_mat,
    curr_beta,
    curr_sigma2
  )
  # Step 2: Update sigma2
  eps_mat <- Y_mat - curr_mu - curr_beta
  a_post_sig <- (N * M) / 2 + prior_ig_a
  b_post_sig <- sum(eps_mat^2) / 2 + prior_ig_b
  curr_sigma2 <- 1 / rgamma(1, shape = a_post_sig, rate = b_post_sig)

  # --- BLOCK UPDATE PREPARATION ---
  Y_star <- Y_mat - curr_mu # SPDE explains the residuals
  B_mat <- Y_star / curr_sigma2
  Q_post_curr <- Q_pr_curr + Diagonal(N, 1 / curr_sigma2)
  chol_post_curr <- chol(Q_post_curr)
  M_post_curr <- solve(chol_post_curr, solve(t(chol_post_curr), B_mat))

  log_det_pr_curr <- 2 * sum(log(diag(chol_pr_curr)))
  log_det_post_curr <- 2 * sum(log(diag(chol_post_curr)))
  quad_curr <- sum(B_mat * M_post_curr)
  log_lik_curr <- 0.5 *
    M *
    log_det_pr_curr -
```

```
  0.5 * M * log_det_post_curr +
  0.5 * quad_curr

# Adaptive Logic
if (g > 200 && (g %% 10 == 0)) {
  emp_cov <- cov(mcmc_history[1:(g - 1), ])
  prop_cov <- scale_factor * emp_cov + diag(1e-4, nrow = 4)
}

# Step 3: Marginal M-H
mh_step <- marginal_metropolis_hastings(
  curr_tau2,
  curr_kappa2,
  curr_gamma,
  curr_theta,
  B_mat,
  curr_sigma2,
  C_mat,
  K11,
  K22,
  K12,
  diag_C,
  prop_cov,
  Q_pr_curr,
  chol_pr_curr,
  chol_post_curr,
  M_post_curr,
  log_lik_curr
)

curr_tau2 <- mh_step$tau2
curr_kappa2 <- mh_step$kappa2
curr_gamma <- mh_step$gamma
curr_theta <- mh_step$theta

Q_pr_curr <- mh_step$Q_pr
chol_pr_curr <- mh_step$chol_pr
chol_post_curr <- mh_step$chol_post
M_post_curr <- mh_step$M_post
if (mh_step$accepted) {
  acc_count <- acc_count + 1
}

# Step 4: Joint Beta update (Anomaly)
Z_mat <- matrix(rnorm(N * M), nrow = N, ncol = M)
curr_beta <- as.matrix(M_post_curr + solve(chol_post_curr, Z_mat))

# Save parameters
samples_sigma2[g] <- curr_sigma2
samples_tau2[g] <- curr_tau2
samples_kappa2[g] <- curr_kappa2
samples_gamma[g] <- curr_gamma
samples_theta[g] <- curr_theta
```

```
  samples_beta1[g] <- curr_beta[1, 1]
  if (g > burnin) {
    beta_sum_post <- curr_beta + beta_sum_post
  }
  if (g %in% keep_iter) {
    beta_draws[, , match(g, keep_iter)] <- curr_beta
  }

  mcmc_history[g, ] <- c(
    log(curr_tau2),
    log(curr_kappa2),
    log(curr_gamma),
    curr_theta
  )
}
end_time <- Sys.time()

cat(sprintf(
  "Elapsed Time: %.2f seconds\n",
  as.numeric(difftime(end_time, start_time, units = "secs"))
))
```

```
## Elapsed Time: 71.66 seconds
```

```
cat(sprintf("Acceptance Rate M-H: %.2f%%\n", (acc_count / G) * 100))
```

```
## Acceptance Rate M-H: 18.36%
```

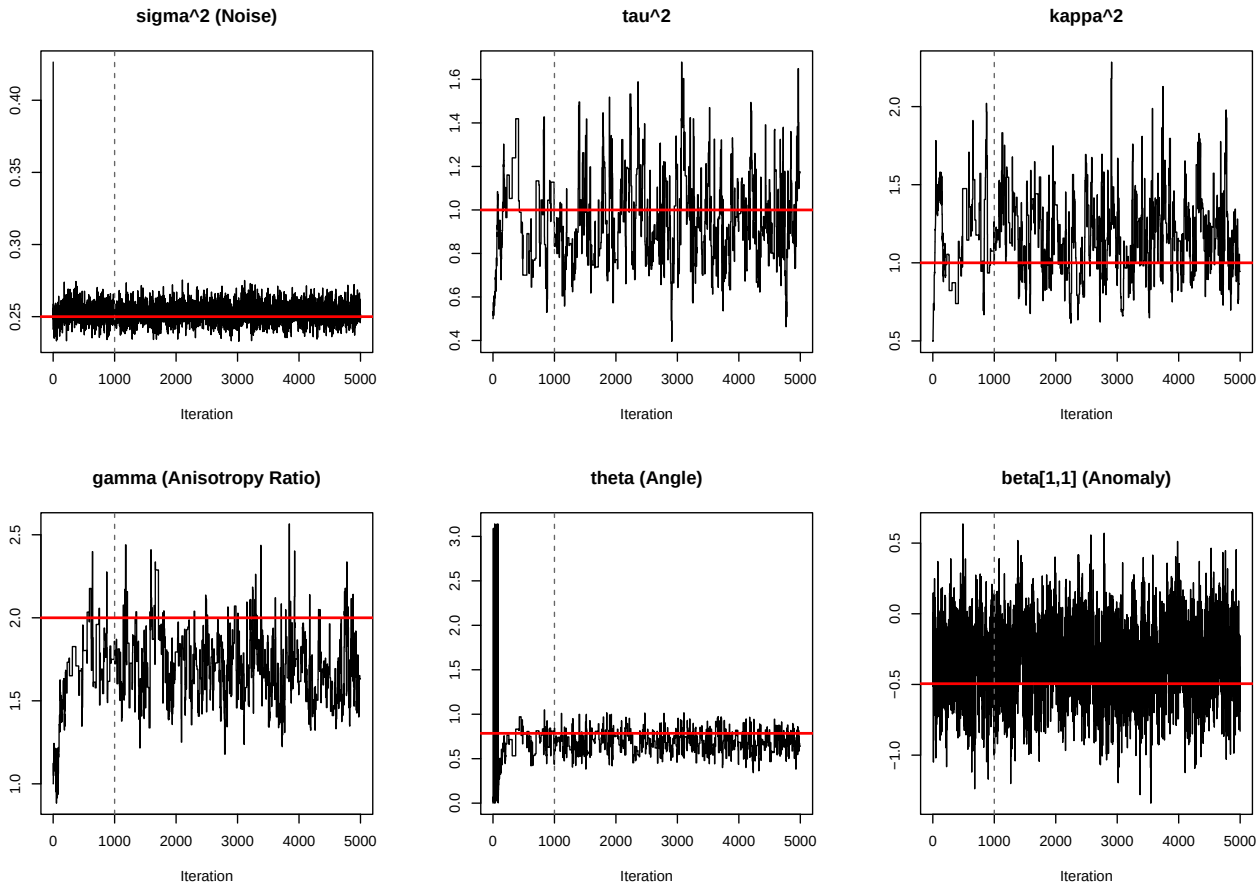
Convergence diagnostics

Trace plots

These show the **full** chain, so you can see the transient phase and judge whether the burn-in (dashed grey line) is long enough. The red line is the true value.

```
trace_list <- list(
  list(x = samples_sigma2, main = "sigma^2 (Noise)", true = true_sigma_e^2),
  list(x = samples_tau2, main = "tau^2", true = true_tau^2),
  list(x = samples_kappa2, main = "kappa^2", true = true_kappa^2),
  list(x = samples_gamma, main = "gamma (Anisotropy Ratio)", true = true_gamma),
  list(x = samples_theta, main = "theta (Angle)", true = true_theta),
  list(x = samples_beta1, main = "beta[1,1] (Anomaly)", true = true_anomaly_mat[1, 1])
)

par(mfrow = c(2, 3))
for (tr in trace_list) {
  plot(tr$x, type = "l", main = tr$main, xlab = "Iteration", ylab = "")
  abline(v = burnin, lty = 2, col = "grey40")
  abline(h = tr$true, col = "red", lwd = 2)
}
```



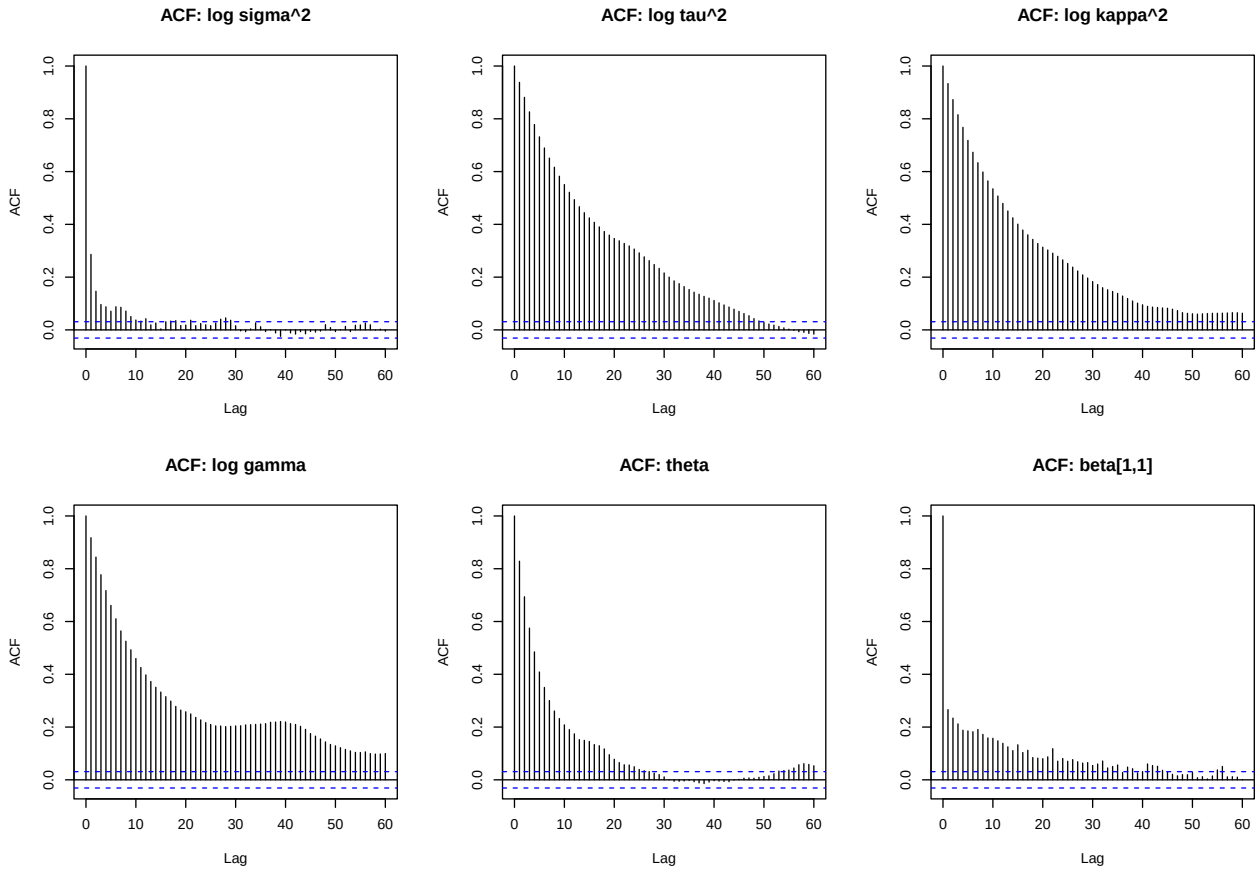

```
par(mfrow = c(1, 1))
```

Autocorrelation and effective sample size

These use the post-burn-in draws only, on the log scale where the Metropolis step moves.

```
post_draws <- cbind(
  "log sigma^2" = log(samples_sigma2[idx_post]),
  "log tau^2"   = log(samples_tau2[idx_post]),
  "log kappa^2" = log(samples_kappa2[idx_post]),
  "log gamma"   = log(samples_gamma[idx_post]),
  "theta"       = samples_theta[idx_post],
  "beta[1,1]"   = samples_beta1[idx_post]
)

par(mfrow = c(2, 3))
for (j in seq_len(ncol(post_draws))) {
  acf(post_draws[, j], lag.max = 60, main = paste("ACF:", colnames(post_draws)[j]))
}
```




```
par(mfrow = c(1, 1))

ess <- coda::effectiveSize(coda::mcmc(post_draws))
knitr::kable(
  data.frame(ESS = round(ess), `out of` = length(idx_post), check.names = FALSE),
  caption = "Effective sample size (post burn-in)"
)
```

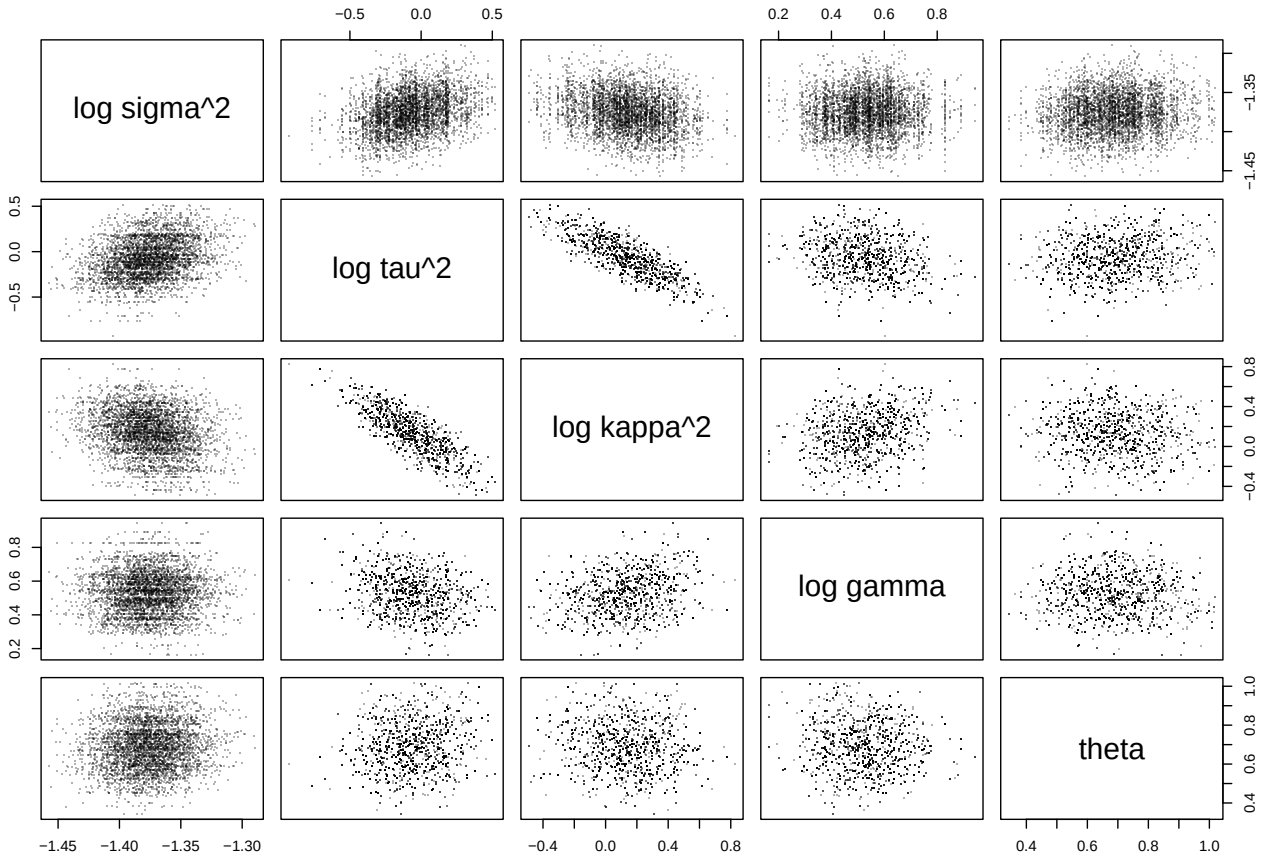
Table 1: Effective sample size (post burn-in)

	ESS	out of
log sigma^2	1214	4000
log tau^2	128	4000
log kappa^2	133	4000
log gamma	172	4000
theta	358	4000
beta[1,1]	476	4000

Posterior correlation between hyperparameters

A strong negative correlation between $\log \tau^2$ and $\log \kappa^2$ is expected: the marginal variance of the field scales like $1/(4\pi\tau^2\kappa^2)$, so the posterior forms a ridge.

```r
pairs(post_draws[, 1:5], pch = ".", col = adjustcolor("black", 0.3))
```




```r
knitr::kable(round(cor(post_draws[, 1:5]), 2), caption = "Posterior correlations")
```

Table 2: Posterior correlations

	log sigma^2	log tau^2	log kappa^2	log gamma	theta
log sigma^2	1.00	0.29	-0.21	0.02	0.08
log tau^2	0.29	1.00	-0.83	-0.23	0.17
log kappa^2	-0.21	-0.83	1.00	0.25	-0.12
log gamma	0.02	-0.23	0.25	1.00	0.01
theta	0.08	0.17	-0.12	0.01	1.00

Posterior estimates vs ground truth

```r
summ <- function(x, true) {
  q <- quantile(x[idx_post], c(0.025, 0.5, 0.975))
  c(True = true, Mean = mean(x[idx_post]), `2.5%` = q[[1]], Median = q[[2]],
    `97.5%` = q[[3]])
}
est_tab <- rbind(
```

```
  "sigma^2" = summ(samples_sigma2, true_sigma_e^2),
  "tau^2"   = summ(samples_tau2, true_tau^2),
  "kappa^2" = summ(samples_kappa2, true_kappa^2),
  "gamma"   = summ(samples_gamma, true_gamma),
  "theta"   = summ(samples_theta, true_theta),
  "beta_11" = summ(samples_beta1, true_anomaly_mat[1, 1])
)
knitr::kable(round(est_tab, 4), caption = "Posterior summaries (post burn-in)")
```

Table 3: Posterior summaries (post burn-in)

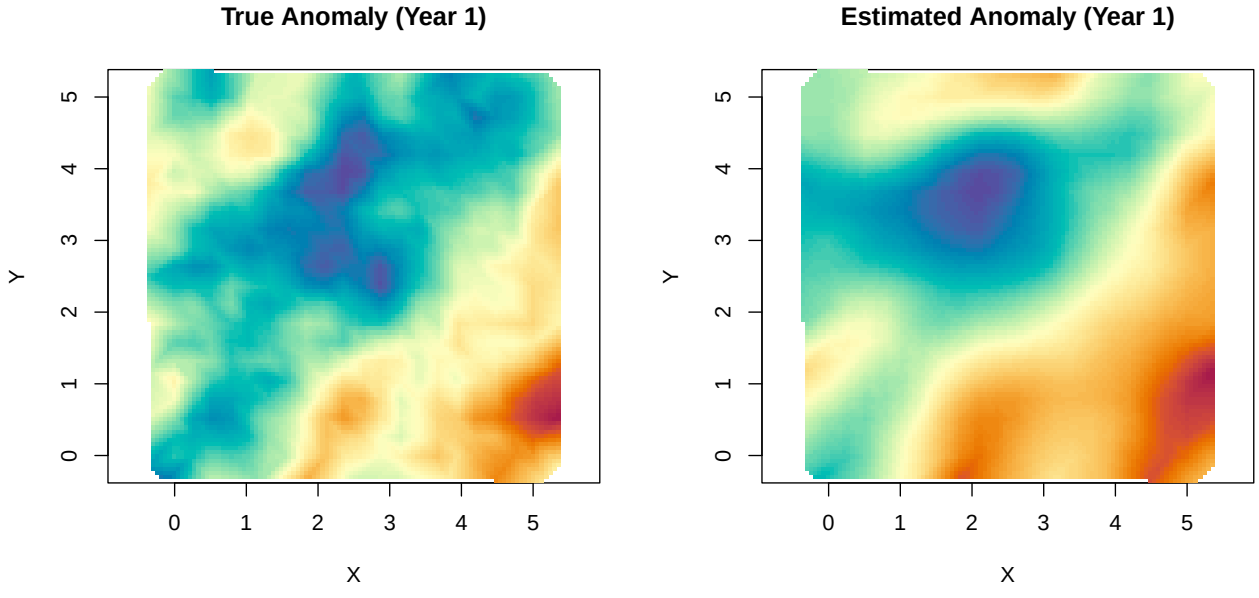
	True	Mean	2.5%	Median	97.5%
sigma^2	0.2500	0.2527	0.2407	0.2526	0.2655
tau^2	1.0000	0.9501	0.6237	0.9224	1.4241
kappa^2	1.0000	1.1819	0.7393	1.1558	1.7487
gamma	2.0000	1.7154	1.3585	1.7105	2.1654
theta	0.7854	0.6877	0.4683	0.6854	0.9297
beta_11	-0.4946	-0.3321	-0.8499	-0.3318	0.1705

Estimated anomaly field

The plot projects vertex values onto a continuous surface using the first-order FEM basis functions.

```
beta_est_mean <- beta_sum_post / (G - burnin)

par(mfrow = c(1, 2))
plot_spde_field(mesh, true_anomaly_mat[, 1], main_title = "True Anomaly (Year 1)")
plot_spde_field(mesh, beta_est_mean[, 1], main_title = "Estimated Anomaly (Year 1)")
```




```
par(mfrow = c(1, 1))
```

Posterior mean and accuracy metrics

For each node i and year t we summarise the draws of $\beta_{i,t}$ by their posterior mean and compare it with the true anomaly. As a reference, we also compute a naive estimate from the data alone: Y_t minus the node's average over the years.

```
# Posterior summaries for each (node, year) from the stored draws
beta_mean <- apply(beta_draws, c(1, 2), mean)
beta_sd <- apply(beta_draws, c(1, 2), sd)

# Errors with respect to the true anomaly
truth <- true_anomaly_mat
err <- beta_mean - truth

rmse <- sqrt(mean(err^2))
mae <- mean(abs(err))
corr <- cor(as.vector(beta_mean), as.vector(truth))

# Naive baseline: the data alone (each year minus the node's average over years)
naive <- Y_mat - rowMeans(Y_mat)
rmse_naive <- sqrt(mean((naive - truth)^2))
mae_naive <- mean(abs(naive - truth))
corr_naive <- cor(as.vector(naive), as.vector(truth))

knitr::kable(
  data.frame(
    Metric = c("RMSE", "MAE", "Correlation with truth"),
    Model = round(c(rmse, mae, corr), 4),
    `Naive baseline` = round(c(rmse_naive, mae_naive, corr_naive), 4),
    check.names = FALSE
  ),
  caption = "Accuracy of the posterior mean of beta"
)
```

Table 4: Accuracy of the posterior mean of beta

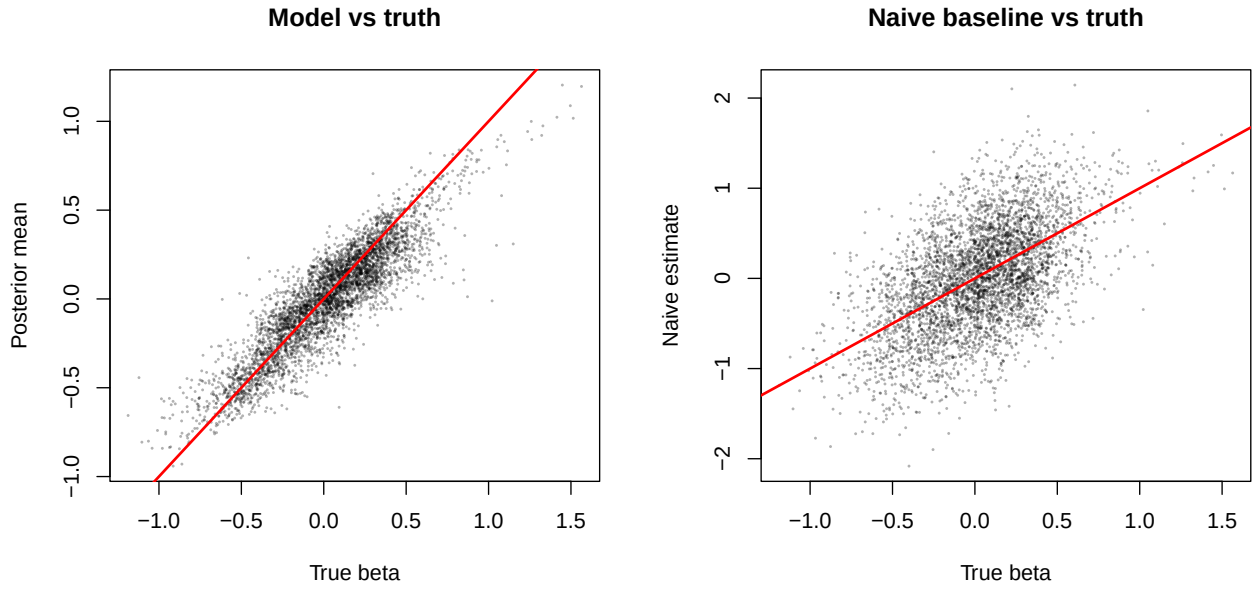
Metric	Model	Naive baseline
RMSE	0.1536	0.4839
MAE	0.1173	0.3853
Correlation with truth	0.8939	0.5483

Estimated vs true values

Points on the red line $y = x$ are estimated exactly. A slope slightly below 1 is expected, because the prior shrinks extreme values towards 0.

```
par(mfrow = c(1, 2))
plot(truth, beta_mean, pch = 16, cex = 0.3, col = adjustcolor("black", 0.3),
     xlab = "True beta", ylab = "Posterior mean", main = "Model vs truth")
abline(0, 1, col = "red", lwd = 2)
plot(truth, naive, pch = 16, cex = 0.3, col = adjustcolor("black", 0.3),
```

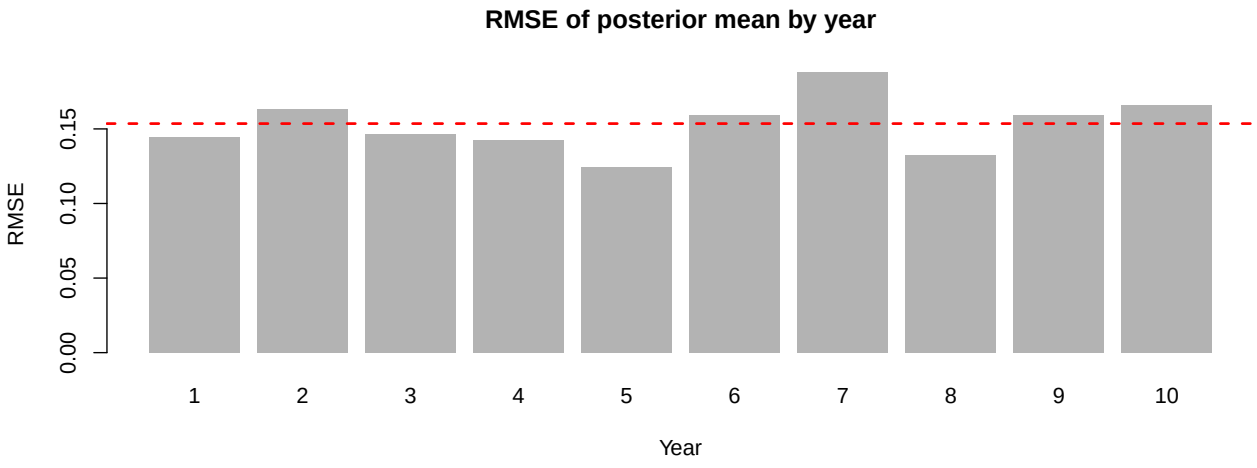
```
     xlab = "True beta", ylab = "Naive estimate", main = "Naive baseline vs truth")
abline(0, 1, col = "red", lwd = 2)
```




```
par(mfrow = c(1, 1))
```

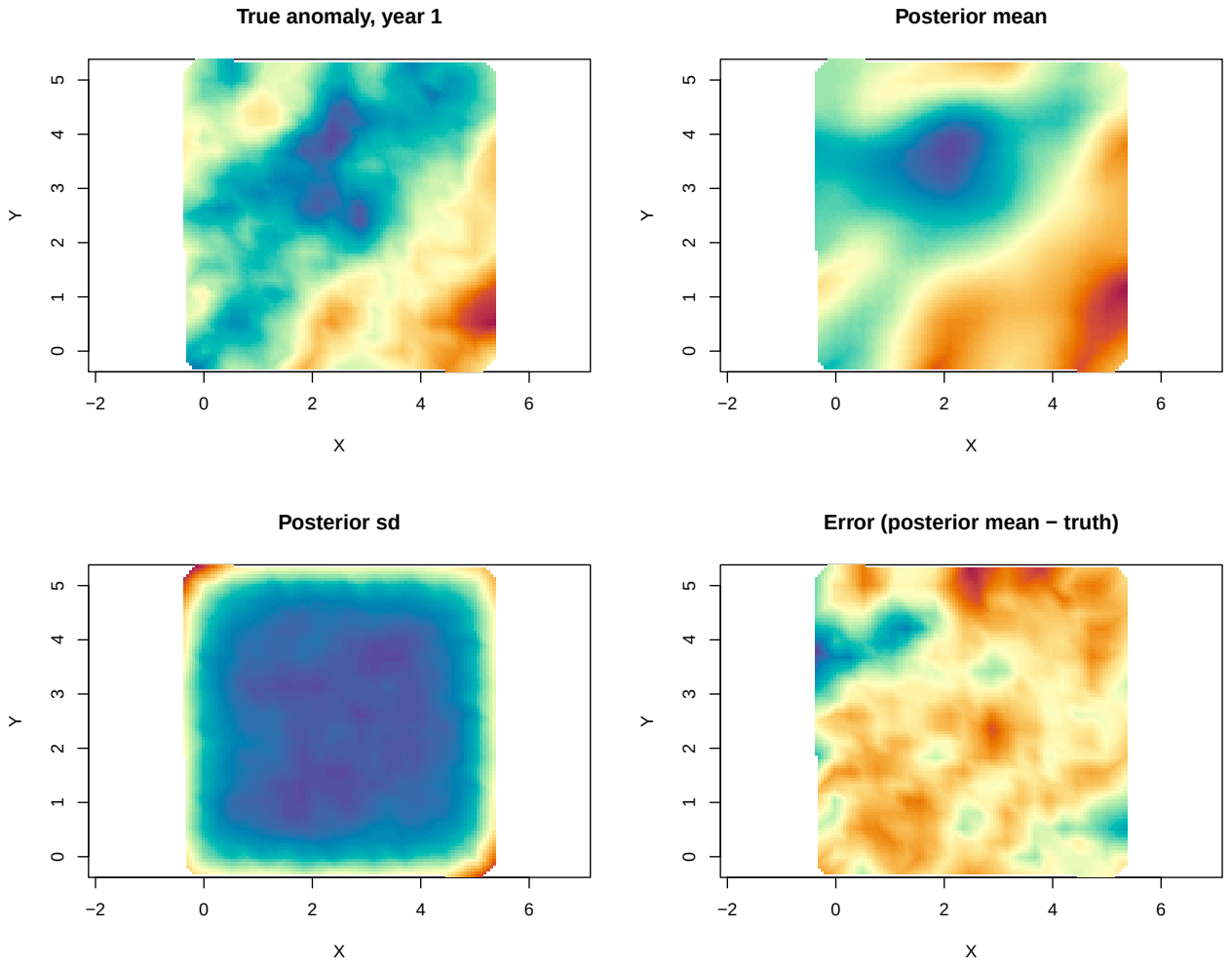
Accuracy by year

```
rmse_year <- sqrt(colMeans(err^2))
barplot(rmse_year, names.arg = seq_len(M), xlab = "Year", ylab = "RMSE",
        main = "RMSE of posterior mean by year", col = "grey70", border = NA)
abline(h = rmse, col = "red", lwd = 2, lty = 2) # overall RMSE
```



Maps for one year

```r
yr <- 1
par(mfrow = c(2, 2))
plot_spde_field(mesh, truth[, yr], main_title = paste("True anomaly, year", yr))
plot_spde_field(mesh, beta_mean[, yr], main_title = "Posterior mean")
plot_spde_field(mesh, beta_sd[, yr], main_title = "Posterior sd")
plot_spde_field(mesh, err[, yr], main_title = "Error (posterior mean - truth)")
```




```r
par(mfrow = c(1, 1))
```